

Automaten

Zeit: 2 Sekunden

Speicherplatz: 1024 MiB

Wir haben ein Feld, welches aus N Zellen in einer Reihe besteht. Die Zellen sind von links nach rechts mit den Zahlen von 0 bis $N - 1$ nummeriert. Alle Zellen haben unterschiedliche Höhen: Die Höhe der Zelle i ist p_i und die Sequenz $p_0, p_1, \dots, p_{N-1}$ ist eine Permutation der Zahlen $0, 1, \dots, N - 1$.

Zwei Zellen mit **Indices** i und j nennen wir *nah* genau dann, wenn $|i - j| \leq 1$ gilt. Insbesondere ist jede Zelle sich selbst *nah*.

Ein Roboter steht auf dem Spielfeld und ist anfänglich auf irgendeiner Zelle platziert. Der Roboter akzeptiert Kommandos. Es gibt zwei Arten von Kommandos:

- **MAX**: Die nächste Position des Roboters ist die Zelle, die unter den Zellen, die *nah* zur aktuellen Position des Roboters sind, die **maximale** Höhe hat.
- **MIN**: Die nächste Position des Roboters ist die Zelle, die unter den Zellen, die *nah* zur aktuellen Position des Roboters sind, die **minimale** Höhe hat.

Da eine Zelle *nah* zu sich selbst ist, kann es natürlich auch passieren, dass der Roboter in seiner aktuellen Zelle bleibt.

Ein *Programm* ist eine endliche Sequenz von Kommandos, bei der jedes Kommando entweder **MAX** oder **MIN** ist. Wenn der Roboter in Zelle X startet und einem Programm S folgt, endet er in einer eindeutig bestimmten Zelle, die wir als $\text{result}(S, X)$ bezeichnen.

Du musst Q Anfragen beantworten. Bei jeder bekommst du eine Menge von K_i Startzellen ($0 \leq i < Q$). Genauer gesagt betrifft die i -te Anfrage einige Zellen $x_0, x_1, \dots, x_{K_i-1}$; der Roboter wird in einer davon platziert. Dir ist im Voraus nicht bekannt, in welcher. Für jede Anfrage musst du bestimmen, ob ein Programm existiert, welches den Roboter zur gleichen Endzelle bewegt, unabhängig davon, welche Startposition aus x verwendet wird.

Formal ausgedrückt musst du bestimmen, ob ein Programm S existiert, welches folgende Bedingung erfüllt:

$$\text{result}(S, x_0) = \text{result}(S, x_1) = \dots = \text{result}(S, x_{K_i-1}).$$

Die Permutation p ist **die gleiche** für alle Anfragen.

Beachte, dass du kein solches Programm erstellen musst – du musst nur angeben, ob ein solches existiert.

Implementierungsdetails

Die erste Funktion, die du implementieren musst, ist die folgende:

```
void initialize(std::vector<int> p)
```

- p : eine Permutation der Zahlen 0 bis $N - 1$.

Die zweite Funktion, die du implementieren musst, ist die folgende:

```
bool exists_program(std::vector<int> x)
```

- x : ein Array, das die Zellen in streng aufsteigender Reihenfolge angibt, gegeben für eine Anfrage.

Die Funktion `initialize` wird genau einmal aufgerufen, und zwar vor allen Aufrufen von `exists_program`.

Die Funktion `exists_program` wird Q Mal aufgerufen, einmal für jede Anfrage. Sie muss `true` zurückgeben, falls ein Programm existiert, mit dem der Roboter in der gleichen Zelle endet, egal von welcher Zelle er startet. Andernfalls muss das Programm `false` zurück geben.

Beschränkungen

- $3 \leq N \leq 2 \cdot 10^5$
- $1 \leq Q \leq 5 \cdot 10^5$
- $p_0, p_1, \dots, p_{N-1}$ ist eine Permutation der Zahlen $0, 1, \dots, N - 1$
- $2 \leq K_i \leq N$
- $0 \leq x_0 < x_1 < \dots < x_{K_i-1} < N$ für jede Anfrage
- Die Summe von K_i über alle Q Anfragen überschreitet nicht 10^6 .

Beispiele

Sample Grader Eingabe 1	Sample Grader Ausgabe 1
3 3 0 2 1 2 0 2 3 0 1 2 2 1 2	111

Sample Grader Eingabe 2	Sample Grader Ausgabe 2
7 3 0 4 2 1 3 5 6 3 0 1 3 3 1 3 4 2 3 6	001

Erklärung von Beispiel 1

In diesem Beispiel ist $p = [0, 2, 1]$.

Für die zweite Abfrage, startet der Roboter entweder in Zelle 0, Zelle 1 oder Zelle 2. Betrachten wir das Programm `[MAX]`.

- Wenn der Roboter in Zelle 0 startet: Die Zellen *nah* an Zelle 0 sind die Zellen mit den Indizes 0 und 1. Wenn der Roboter das Kommando `MAX` ausführt, bewegt er sich zu Zelle 1, da $p_0 < p_1$.
- Wenn der Roboter in Zelle 1 startet: Die Zellen *nah* an Zelle 1 sind die Zellen mit den Indizes 0, 1 und 2. Wenn der Roboter das Kommando `MAX` ausführt, bleibt er in Zelle 1, da $p_0 < p_1$ und $p_1 > p_2$.
- Wenn der Roboter in Zelle 2 startet: Die Zellen *nah* an Zelle 2 sind die Zellen mit den Indizes 1 und 2. Wenn der Roboter das Kommando `MAX` ausführt, bewegt er sich zu Zelle 1, da $p_1 > p_2$.

Daher existiert ein Programm, nach dem der Roboter von allen drei Startzellen aus in Zelle 1 endet. Die Antwort auf diese Anfrage ist somit `true`. Andere Programme funktionieren auch, wie zum Beispiel `[MIN, MAX]`, `[MIN, MIN, MAX, MIN, MAX, MIN]` und `[MAX, MIN, MAX]`.

Erklärung von Beispiel 2

In diesem Beispiel ist $p = [0, 4, 2, 1, 3, 5, 6]$.

Für die ersten beiden Anfragen kann gezeigt werden, dass kein Programm existiert, bei dem der Roboter in der gleichen Zelle endet für alle gegebenen Startzellen. Die Antwort auf diese Anfragen ist somit `false`.

Betrachten wir die letzte Anfrage, bei der der Roboter in Zelle 3 oder Zelle 6 starten kann. Betrachten wir das Programm `[MAX, MAX, MAX]`.

- Wenn der Roboter in Zelle 3 startet: Die Zellen *nah* an Zelle 3 sind die Zellen mit den Indizes 2, 3 und 4. Wenn der Roboter das Programm ausführt, bewegt er sich zuerst zu Zelle 4, da $p_3 < p_4$ and $p_2 < p_4$. Den nächsten beiden Kommandos im Programm ebenso folgend, endet der Roboter in Zelle 6.
- Wenn der Roboter in Zelle 6 startet: Der Roboter bleibt in Zelle 6 und endet dort.

Die Antwort auf diese Anfrage ist somit `true`. Auch das Programm `[MIN, MAX, MAX, MAX]` funktioniert. Das muss allerdings nicht für alle Programme stimmen. Ist das Programm zum Beispiel `[MAX, MAX]`, endet der Roboter in Zelle 5, wenn er in Zelle 3 startet und in Zelle 6, wenn er in Zelle 6 startet.

Teilaufgaben

Teilaufgabe	Punkte	N	Q	K_i	Zusätzliche Beschränkungen
0	0	–	–	–	Die Beispiele.
1	3	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$	$= 2$	Wenn die Antwort auf eine Anfrage positiv ist, dann existiert ein valides Programm, welches aus genau einem Kommando besteht.
2	7				Wenn die Antwort auf eine Anfrage positiv ist, dann existiert ein valides Programm, welches aus maximal 5 Kommandos besteht.
3	9	≤ 100	≤ 500	$= 2$	–
4	17	$\leq 5\,000$	$\leq 5 \cdot 10^5$	$= 2$	–
5	7	$\leq 2 \cdot 10^5$			$x_1 = x_0 + 1$ für jede Anfrage.
6	8	$\leq 5\,000$	$\leq 5 \cdot 10^5$	$\leq N$	–
7	13	$\leq 2 \cdot 10^5$	$\leq 5 \cdot 10^5$	$\leq N$	Die Permutation ist erst aufsteigend, dann absteigend, dann wieder aufsteigend. Formal ausgedrückt existieren genau zwei Ganzzahlen a, b so, dass $0 < a < b < N - 1$ und $p_0 < p_1 < \dots < p_a > p_{a+1} > \dots > p_b < p_{b+1} < \dots < p_{N-1}$.
8	13				$p_0 < p_1 > p_2 < p_3 > \dots < p_{N-1}$ wenn N gerade ist, oder $p_0 < p_1 > p_2 < p_3 > \dots > p_{N-1}$ wenn N ungerade ist.
9	23				–

Sample Grader

Das Eingabeformat ist das folgende:

- Zeile 1: Zwei Ganzzahlen – die Werte von N und Q ;

- Zeile 2: N Ganzzahlen $p_0, p_1, \dots, p_{N-1}$, wobei p_i die Höhe der i -ten Zelle ist;
- Zeile $3 + i$: Die Zellen für die i -te Anfrage – eine Ganzzahl K_i und in der gleichen Zeile K_i Ganzzahlen $x_0, x_1, \dots, x_{K_i-1}$, welche die möglichen Startpunkte des Roboters bezeichnen.

Das Aufgabeformat ist das folgende:

- Zeile 1: Ein binärer *String* der Länge Q . Das i -te Zeichen ist 1, wenn die Antwort auf die i -te Anfrage `true` ist, und 0 sonst.